

Scalable Big Data Architecture for Real-Time Industrial Anomaly Detection in Multivariate Time-Series Under a Sub-20 ms SLA

Mohtashim Ali

[Department / School], [University Name]
[email address]

Usman Butt

[Department / School], [University Name]
[email address]

Abstract

Modern industrial systems generate massive volumes of continuous multivariate time-series data from hundreds of sensors simultaneously, in which latent equipment defects are masked by overlapping sensor patterns. Existing analytical frameworks struggle to reconcile two conflicting engineering requirements: processing high-velocity periodic streams with minimal computational overhead, and achieving low-latency anomaly alerting. This paper presents a scalable pipeline architecture for real-time anomaly detection in industrial multivariate time series under a strict sub-20 ms service-level agreement (SLA). The architecture couples a multi-threaded, back-pressure-aware streaming ingestion layer with two interchangeable analytical backends: a reconstruction-based LSTM autoencoder, and a fast-initializing mathematical backend combining seasonal-trend decomposition (STL) with a closed-form Dynamic Mode Decomposition (DMD) operator. Both backends feed a common scoring node that fuses localized reconstruction/prediction error with the harmonic mean of Euclidean distance to reference operating-regime centroids. We evaluate both backends on the NASA C-MAPSS turbofan degradation benchmark and the SWaT secure water treatment dataset, measuring detection accuracy (F1, AUCROC) and per-vector scoring latency against the 20 ms SLA under both steady-state and burst throughput conditions. [TBD: one to two sentences summarizing the headline empirical findings once Phase 4 benchmarks have been run.]

Index Terms— *anomaly detection, multivariate time series, industrial IoT, streaming architecture, autoencoders, dynamic mode decomposition, AIOPS, low-latency systems*

1 Introduction

Modern industrial systems are highly interconnected, generating continuous multivariate time-series (MTS) streams from hundreds of sensors simultaneously. Operational complexity frequently masks latent equipment defects, leading to unexpected system breakdowns and expensive operational downtime. Detecting anomalous behavior in these streams quickly enough to act on it is a distinct engineering problem from detecting it accurately: a model that is highly accurate

but too slow to keep pace with the incoming stream is not deployable in a real-time industrial control loop.

Current analytical frameworks fail to reconcile two conflicting engineering needs: processing massive periodic streams with minimal computational overhead, and achieving low-latency alerting [1, 2]. Deep reconstruction-based models such as LSTM autoencoders offer strong detection accuracy on complex multi-sensor data [3], but impose per-inference compute costs and prolonged initialization periods that are difficult to sustain in fast-paced production ecosystems. Conversely, lightweight statistical methods initialize quickly and score cheaply, but have historically been overwhelmed by the volume, velocity, and periodicity of industrial streams.

This paper addresses that tension directly. We architect, implement, and evaluate an end-to-end pipeline for anomaly detection in industrial MTS data under a strict sub-20 ms SLA, defined as the elapsed time from a sensor vector leaving the ingestion queue to an anomaly score being produced. The contributions are threefold:

1. **An end-to-end, reproducible pipeline** spanning offline model development, multi-threaded streaming ingestion with explicit back-pressure semantics, and SLA-aware per-vector latency instrumentation.
2. **A direct empirical comparison** of a deep reconstruction-based backend (LSTM autoencoder) against a closed-form mathematical backend (STL + DMD) on identical data, identical streaming infrastructure, and identical evaluation criteria — isolating the accuracy/latency trade-off from confounding pipeline differences.
3. **An elasticity analysis** of both backends under sudden throughput spikes, measuring drop rate and tail latency beyond steady-state benchmarking alone.

The remainder of this paper is organized as follows. Section II reviews related work. Section III describes the system architecture. Section IV details the experimental methodology. Section V presents results, and Section VI concludes.

2 Related Work

2.1 AIOps and Industrial MTS Anomaly Detection

The reliance of modern industrial plants on automated machinery has made MTS anomaly detection vital for infrastructure reliability. In dense industrial settings, anomalies are hidden within overlapping sensor patterns, such as spatial correlations between vibration, pressure, and temperature metrics. Traditional statistical methods are frequently overwhelmed by the volume, velocity, and periodic nature of these streams, producing delayed alerts or elevated false-alarm rates [1, 2]. Entropy-based sequence characterization [4] established early foundations for detecting irregularity in physiological and mechanical time series, but does not by itself address the throughput requirements of modern industrial deployments.

2.2 Reconstruction-Based Deep Learning

Reconstruction-based neural models detect anomalies by learning a compressed latent representation of normal operational cycles and flagging inputs that reconstruct poorly. LSTM-based encoder-decoder architectures extend this to sequential data by modeling temporal dependencies within each window [3], and have proven effective in complex multi-sensor industrial fields. However, deep models present structural challenges in production ecosystems: prolonged training and initialization periods, and per-inference costs that scale poorly against tight latency budgets on commodity hardware [2].

2.3 Fast-Initializing Mathematical Alternatives

A complementary line of work favors closed-form mathematical methods over gradient-trained models, prioritizing rapid initialization and low, predictable inference latency. Seasonal-trend decomposition via Loess (STL) [6] separates a series into trend, seasonal, and residual components, allowing anomalies to surface as elevated residual energy relative to a learned baseline. Dynamic Mode Decomposition (DMD) [7] fits a best-fit linear operator approximating one-step-ahead dynamics through a single truncated singular value decomposition, requiring no iterative training. Recent systems combining these primitives — exemplified by the VersaGuardian model¹ — report competitive detection accuracy at initialization and latency costs far below deep alternatives.

¹[TBD: the VersaGuardian model (2026) is described in the project proposal’s background research with reported figures of approximately 20-minute initialization and 15.28 ms average detection latency; a citable primary source must be located and inserted here before submission. These figures should not be treated as established fact until independently verified.]

[TBD: architecture diagram — sensor window → parallel Autoencoder / fast STL+DMD branches → shared scoring node → anomaly decision. Export from project documentation.]

Figure 1: End-to-end pipeline architecture. A single sensor window is scored independently by each backend; both feed the same scoring node, isolating the backend comparison from pipeline confounds.

2.4 Research Gap

Despite these methodological innovations, a persistent gap remains in coupling fast detection algorithms with full-scale streaming data architectures. An algorithm is only as useful as the pipeline supplying its data: industrial platforms require structured streaming ingestion that feeds high-throughput streams into an evaluation engine without introducing serialization or I/O bottlenecks. Direct empirical comparisons of deep and closed-form backends under an explicit real-time SLA — accounting for queueing mechanics, back-pressure, and throughput-spike elasticity rather than offline batch accuracy alone — remain comparatively rare. This work addresses that gap.

3 System Architecture

The pipeline is organized into three programmatic stages: an ingestion layer, a processing and analytics core, and an evaluation matrix (scoring node). Fig. 1 summarizes the data flow.

3.1 Ingestion Layer

The ingestion layer is a multi-threaded messaging-queue simulation in Python. A producer thread reads pre-scaled sensor arrays and pushes multivariate windows into a bounded, thread-safe queue at a configurable production rate, simulating an industrial message bus. The bounded queue makes producer back-pressure an explicit, observable property of the system. Two overflow policies are supported: blocking (back-pressure propagates to the producer) for steady-state operation, and drop-on-full for elasticity testing, simulating a broker’s overflow behavior under a sudden throughput spike.

3.2 Processing and Analytics Core

A consumer thread pulls vectors from the queue and scores them one at a time — deliberately unbatched, so that measured latency reflects genuine single-vector inference cost rather than amortized batch throughput. Two interchangeable analytical backends are evaluated:

3.2.1 LSTM Autoencoder

A reconstruction-based encoder–decoder network trained exclusively on normal-operation windows with early stopping on a held-out normal validation split. The encoder compresses each window through stacked LSTM layers into a latent vector; the decoder reconstructs the window from the latent seed. At inference, the mean squared reconstruction error over the window serves as the raw anomaly signal.

3.2.2 Fast Mathematical Backend (STL + DMD)

The fast backend combines two closed-form components fit once on normal data: (i) a per-channel STL residual-energy baseline, capturing the expected trend/seasonal signature of healthy operation; and (ii) a DMD linear operator A such that $x_{t+1} \approx Ax_t$, obtained via a single rank-truncated SVD over stacked snapshot pairs. At inference, the operator’s one-step-ahead prediction error, normalized by its baseline error distribution on normal data, serves as the raw anomaly signal. Neither component requires gradient-based training, yielding near-instant initialization relative to the autoencoder.

3.3 Evaluation Matrix (Scoring Node)

Both backends’ raw error signals are z -normalized against normal-data baseline statistics and linearly combined with a multivariate distance term: the harmonic mean of Euclidean distances between each in-window reading and a set of k reference “normal operating regime” centroids obtained by k -means over normal training vectors. The harmonic mean is used deliberately rather than the arithmetic mean: because it is dominated by the smallest distances, a window’s distance score remains low only if its readings sit close to every relevant reference regime, so a single nearby centroid cannot mask a genuinely anomalous vector. The combined score for window w with raw backend error $e(w)$ is

$$s(w) = \alpha \tilde{e}(w) + (1 - \alpha) \widetilde{d_{\text{hm}}}(w), \quad (1)$$

where $\tilde{\cdot}$ denotes z -normalization against normal-data statistics and d_{hm} is the harmonic-mean distance. The detection threshold is calibrated as a high percentile of the normal validation split’s own score distribution, requiring no anomaly labels.

4 Experimental Methodology

4.1 Datasets

Two publicly available industrial benchmarks are used. NASA C-MAPSS [5] provides multivariate sensor telemetry across the run-to-failure lifetime of simulated turbofan engines; we recast it for anomaly detection by labeling the final N cycles before failure of each test unit as anomalous, with healthy training trajectories serving as the normal baseline. SWaT [8] is collected from a fully operational, scaled-down

secure water treatment testbed and provides natively labeled process-control attacks; its attack file serves as the labeled test set and its normal-operation file as the training baseline.

4.2 Preprocessing

Missing values are handled by forward/backward fill with a median fallback. Continuous sensor metrics are normalized with MinMax scaling fit *exclusively* on normal-operation data, preventing anomalous-region statistics from leaking into normalization. Cleaned readings are structured into overlapping contiguous windows, the atomic unit scored by both backends. A window inherits the anomalous label if any reading within it is anomalous — a conservative policy favoring recall on partial-window anomalies.

4.3 SLA Instrumentation

A high-precision monotonic timer wraps exactly the span from a vector leaving the ingestion queue to its anomaly score being produced, per vector. Queue waiting time is deliberately excluded: it is a property of the ingestion/back-pressure subsystem, whereas the 20 ms SLA targets the scoring engine’s own computational cost. We report mean, median, 95th-, and 99th-percentile latency, and the fraction of vectors violating the SLA.

4.4 Steady-State and Stress Protocols

In the steady-state protocol, the producer emits vectors at a fixed target rate through a large bounded queue with blocking overflow; both backends process the identical stream. In the stress protocol, the producer emits unthrottled bursts into a deliberately small queue with drop-on-full overflow, and we report effective throughput, drop rate, queue high watermark, and tail latency — quantifying each backend’s elasticity under load beyond its steady-state envelope.

4.5 Metrics

Detection accuracy is reported as F1-score and area under the receiver operating characteristic curve (AUCROC) at window level. Computational efficiency is reported as the per-vector latency statistics above, tracked against the 20 ms SLA threshold.

5 Results and Discussion

5.1 Detection Accuracy and Steady-State Latency

Table 1 reports accuracy and latency for both backends on both datasets under the steady-state protocol. [TBD: populate from `metrics.json` and `benchmark.report.json` after running `train.offline.py` and `benchmark.py` on

the downloaded datasets. Then discuss: (i) which backend achieves higher F1/AUCROC per dataset; (ii) mean and tail latency of each backend against the 20 ms SLA; (iii) whether the accuracy gap, if any, justifies the latency gap.]

5.2 Elasticity Under Throughput Spikes

Table 2 reports the stress-protocol results. [TBD: populate and discuss which backend degrades more gracefully — in particular whether the fast backend’s lower per-vector compute translates into a materially lower drop rate at identical queue size and burst intensity, and how tail latency shifts under saturation.]

5.3 Threats to Validity

Three limitations qualify the results. First, C-MAPSS anomaly labels are derived heuristically (final N cycles before failure) rather than being natively point-labeled as in SWaT; this labeling policy affects absolute F1 values and cross-dataset comparability. Second, the streaming architecture is a single-machine multi-threaded simulation rather than a distributed deployment; absolute latency figures may not transfer to production hardware without re-benchmarking. Third, the fast backend is a good-faith reconstruction of the STL+DMD approach described in prior work rather than a validated reference implementation.

6 Conclusion and Future Work

This paper presented an end-to-end streaming architecture for real-time industrial MTS anomaly detection under a sub-20 ms SLA, directly comparing a deep LSTM autoencoder against a closed-form STL+DMD backend on identical data and infrastructure. [TBD: one to two sentences stating the headline empirical conclusion once results are in — e.g., whether the fast backend meets the SLA with materially more headroom, and at what accuracy cost, if any.]

Future work includes: learning an adaptive weighting α in (1) rather than fixing it; validating the latency findings on a distributed ingestion substrate (e.g., Kafka) beyond the single-machine simulation; and investigating a hybrid mode that ensembles both backends’ scores where the combined cost still fits the SLA budget.

References

[1] L. Tao, M. Ma, S. Zhang, and D. Pei, “Real-time anomaly detection for large-scale network devices,” *IEEE Transactions on Networking*, vol. 33, no. 3, pp. 1326–1337, Jun. 2025.

[2] T. P. Exarchos *et al.*, “Anomaly detection in industrial plants using neural network architectures,” *IEEE Trans-*

actions on Industrial Informatics, vol. 17, no. 8, pp. 5432–5441, Aug. 2021.

- [3] P. Malhotra, L. Vig, G. Shroff, and P. Agarwal, “Long short term memory networks for anomaly detection in time series,” in *Proc. European Symposium on Artificial Neural Networks (ESANN)*, 2015, pp. 89–94.
- [4] J. S. Richman and J. R. Moorman, “Physiological time-series analysis using approximate entropy and sample entropy,” *American Journal of Physiology-Heart and Circulatory Physiology*, vol. 278, no. 6, pp. H2039–H2049, Jun. 2000.
- [5] A. Saxena, K. Goebel, D. Simon, and N. Eklund, “Damage propagation modeling for aircraft engine run-to-failure simulation,” in *Proc. 1st International Conference on Prognostics and Health Management (PHM)*, 2008, pp. 1–9.
- [6] R. B. Cleveland, W. S. Cleveland, J. E. McRae, and I. Terpenning, “STL: A seasonal-trend decomposition procedure based on loess,” *Journal of Official Statistics*, vol. 6, no. 1, pp. 3–73, 1990.
- [7] P. J. Schmid, “Dynamic mode decomposition of numerical and experimental data,” *Journal of Fluid Mechanics*, vol. 656, pp. 5–28, 2010.
- [8] J. Goh, S. Adepu, K. N. Junejo, and A. Mathur, “A dataset to support research in the design of secure water treatment systems,” in *Proc. International Conference on Critical Information Infrastructures Security (CRITIS)*, 2016.

Table 1: Detection Accuracy and Steady-State Latency (SLA = 20 ms). [TBD: fill from pipeline outputs.]

Dataset	Backend	F1	AUCROC	Mean lat. (ms)	p99 lat. (ms)	SLA viol. (%)
C-MAPSS	LSTM Autoencoder	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
C-MAPSS	Fast (STL+DMD)	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
SWaT	LSTM Autoencoder	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
SWaT	Fast (STL+DMD)	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]

Table 2: Stress/Elasticity Results Under Unthrottled Burst Ingestion. [TBD: fill from pipeline outputs.]

Dataset	Backend	Throughput (vec/s)	Drop rate (%)	Queue HWM	p99 lat. (ms)
C-MAPSS	LSTM Autoencoder	[TBD]	[TBD]	[TBD]	[TBD]
C-MAPSS	Fast (STL+DMD)	[TBD]	[TBD]	[TBD]	[TBD]
SWaT	LSTM Autoencoder	[TBD]	[TBD]	[TBD]	[TBD]
SWaT	Fast (STL+DMD)	[TBD]	[TBD]	[TBD]	[TBD]